# 8. Satisfying Non-Functional Requirements

The previous architecture successfully demonstrated:

- Payment lifecycle
- Webhooks
- Payouts
- Ledger updates
- Deduplication

However, it only works reliably for:

- Small traffic
- Limited concurrency
- Minimal failures

Real-world payment gateways such as Stripe must support:

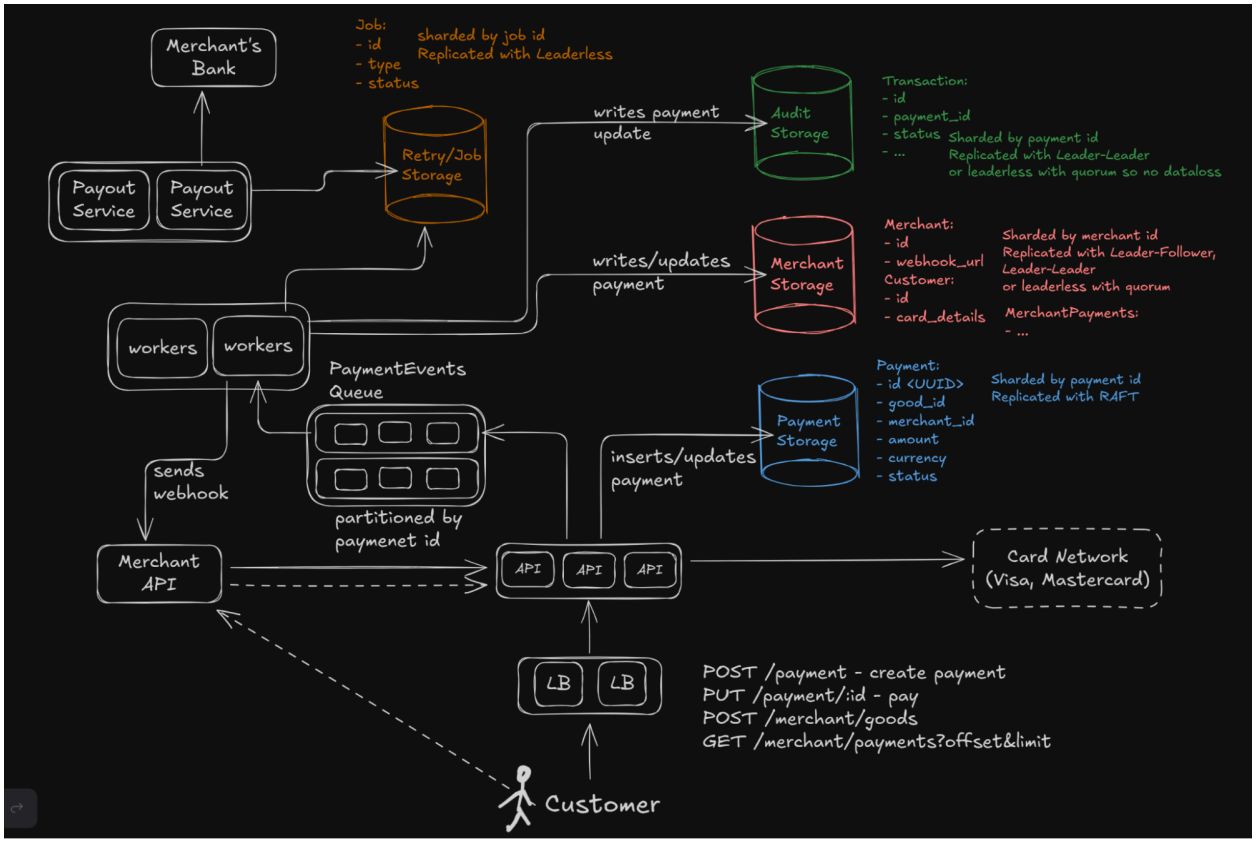
| **Metric** | **Scale** |
|---|---|
| Payment Requests | 500M/day |
| Peak Payment Throughput | 10k RPS |
| Ledger Events | 5B/day |
| Peak Ledger Writes | 100k RPS |
| Merchants | 2M+ |

To operate at this scale, the architecture must evolve into:

A distributed, fault-tolerant, event-driven payment platform.

# 8.1 Distributed Payment Gateway Architecture

## Scalable Distributed Architecture




# 8.2 Architectural Goals

The new architecture is designed to achieve:

- Horizontal scalability
- High availability

- Strong consistency for payments
- Eventual consistency for secondary systems
- Durable ledger writes
- Retry-safe operations
- Fault tolerance
- Asynchronous processing

# 8.3 High-Level Architecture Components

The distributed architecture introduces several new components.

## 8.3.1 Load Balancers (LB)

Incoming traffic first reaches load balancers.

Responsibilities:

- Traffic distribution
- Failover
- Health checks
- DDoS mitigation
- TLS termination

Benefits:

- No single API bottleneck
- Horizontal scaling support

# 8.3.2 Stateless Payment APIs

Multiple Payment API instances run in parallel.

Responsibilities:

- Accept payment requests
- Validate input

- Interact with Card Networks
- Persist payment state
- Publish payment events

Key design principle:

```
None
APIs remain stateless
```

Why stateless?

- Easy scaling
- Fast failover
- Simpler deployment

# 8.3.3 Payment Storage

Stores:

- Payment entities
- Current payment state

Example schema:

| **Field** | **Description** |
| --- | --- |
| payment_id | Unique identifier |
| merchant_id | Merchant reference |
| amount | Payment amount |

| | |
|---|---|
| currency | Currency |
| status | Current payment state |

# Scaling Strategy

## Sharding

Payment data is:

```
None
Partitioned by payment_id
```

This distributes load across multiple database shards.

Benefits:

- Higher write throughput
- Better parallelism
- Reduced contention

# Replication

Payment storage uses replication for:

- High availability
- Disaster recovery
- Read scaling

Possible replication models:

- Leader-Follower
- Multi-Leader
- Leaderless

# 8.3.4 Merchant Storage

Stores:

- Merchant metadata
- Webhook URLs
- Configuration
- Payment methods

## Merchant Storage Scaling

Merchant data is:

```
None
Sharded by merchant_id
```

Replication strategies:

- Leader-Follower
- Multi-Leader
- Leaderless with quorum

Merchant data can tolerate:

```
None
Eventual consistency
```

because temporary staleness is acceptable.

# 8.3.5 Audit (Ledger) Storage

The ledger is the most critical component.

Stores:

- Immutable financial events

- Transaction history
- State transitions

## Ledger Scaling Strategy

Ledger data is:

```
None
Partitioned by payment_id
```

Replication:

- Leader-Leader
- Leaderless quorum replication

Goal:

```
None
No data loss
```

## Why Strong Durability Matters

Ledger correctness must survive:

- Machine failures
- Region failures
- Network partitions

Because:

```
None
Financial history cannot be lost
```

# 8.3.6 Payment Events Queue

A major improvement is introducing:

> Asynchronous event-driven processing

Payment API publishes events to:

```
None
PaymentEvents Queue
```

## Queue Partitioning

Events are:

```
None
Partitioned by payment_id
```

Why?

- Preserve ordering per payment
- Avoid concurrent processing conflicts

Example ordering:

```
None
NEW

→ AUTHORIZED

→ CAPTURED

→ SETTLED
```

must remain sequential.

# 8.3.7 Worker Fleet

Background workers consume events from the queue.

Responsibilities:

- Send webhooks
- Update ledger
- Trigger payouts
- Retry failed operations
- Execute async workflows

## Benefits of Workers

Workers decouple:

- User-facing APIs
- Slow background tasks

Benefits:

- Improved API latency
- Better fault isolation
- Easier retries
- Independent scaling

# 8.3.8 Retry / Job Storage

Used for:

- Durable retries
- Workflow persistence
- Delayed jobs
- Failed operation recovery

Example jobs:

- Webhook retry
- Payout retry

- Reconciliation task

## Scaling Strategy

Jobs are:

```
Sharded by job_id
```

Replication:

```
Leaderless replication
```

Why?

- Retry systems require high availability
- Temporary duplicates are acceptable

# 8.3.9 Payout Service

Payout Service now becomes:

- Distributed
- Retry-safe
- Batch-oriented

Responsibilities:

- Aggregate merchant balances
- Transfer money to merchant banks
- Reconcile bank records

# 8.4 Event-Driven Architecture Benefits

The biggest architectural shift is:

> Moving from synchronous workflows to asynchronous event processing.

## Previous Architecture Problem

Previously:

- API directly performed everything
- Failures caused cascading issues

Now:

- APIs publish events
- Workers process independently

## Advantages

| **Benefit** | **Description** |
|---|---|
| Loose coupling | Services independent |
| Better scalability | Independent scaling |
| Fault tolerance | Queue buffers spikes |
| Retries | Easier background recovery |

| | |
|---|---|
| Higher throughput | Async processing |

# 8.5 Consistency Model

Different parts of the system require different consistency guarantees.

## Strong Consistency Required

Critical flows:

- Payment authorization
- Payment capture
- Deduplication

Need:

```
None
Linearizability
```

because:

```
None
Double charging is unacceptable
```

## Eventual Consistency Allowed

Safe for:

- Merchant metadata
- Analytics
- Webhook delivery status

- Reporting systems

This improves:

- Availability
- Scalability

# 8.6 Fault Tolerance Improvements

The distributed design now tolerates:

- API crashes
- Worker crashes
- Queue failures
- Retry storms
- Database replica failures

## Failure Recovery

Failures are handled using:

- Durable queues
- Retries
- Idempotency
- Replication
- Reconciliation

# 8.7 Scalability Improvements

The architecture now scales horizontally:

| **Component** | **Scaling Strategy** |
| --- | --- |

| | |
|---|---|
| APIs | Add more stateless instances |
| Workers | Add more consumers |
| Databases | Sharding |
| Queues | Partitioning |
| Ledger | Distributed replication |

# 8.8 Availability Improvements

High availability achieved through:

- Replication
- Failover
- Load balancing
- Multi-instance deployment

No single machine failure should stop:

```
None
Payment processing
```

# 8.9 Key Architectural Insight

The final architecture transforms the payment gateway into:

A distributed event-driven financial processing platform.

Core design principles:

- Stateless APIs
- Durable queues
- Immutable ledger
- Async workflows
- Idempotent operations
- Partitioned scalability
- Strong consistency where required

This architecture can now realistically support:

- Millions of merchants
- Billions of ledger events
- Massive seasonal traffic spikes
- Reliable financial correctness at scale

# 8.10 Introducing Kafka for Side Effects

Before addressing deeper non-functional concerns such as:

- reliability,
- scalability,
- fault tolerance,
- and recovery,

we first introduce a critical architectural improvement:

Decoupling side effects from the main payment flow using a message broker.

In large-scale payment systems, a successful payment triggers many additional operations that are independent from the core payment processing logic.

# 8.10.1 Problem With Synchronous Side Effects

When a customer completes a payment, several additional tasks must occur:

- Send webhook to merchant
- Store ledger/audit event
- Trigger notifications
- Update analytics systems
- Populate derived storage/indexes
- Trigger fraud/risk pipelines
- Start payout workflows

These are called:

```
None
Side Effects
```

# Current Problem

In the previous design:

- Payment API directly performed these operations

This creates several issues:

| **Problem** | **Impact** |
| --- | --- |
| Slow webhook endpoint | Increases payment latency |
| Audit storage outage | Payment flow may fail |
| Notification failure | Cascading retries |
| Tight coupling | Hard scalability |

| | |
|---|---|
| Partial failures | Inconsistent system state |

# 8.10.2 Key Observation

These side effects are:

- Independent
- Asynchronous
- Retryable
- Eventually consistent

Therefore:

> They should not block the critical payment path.

# 8.10.3 Introducing a Message Broker (Kafka)

To solve this, we introduce:

> A distributed event streaming/message broker system.

Typically:

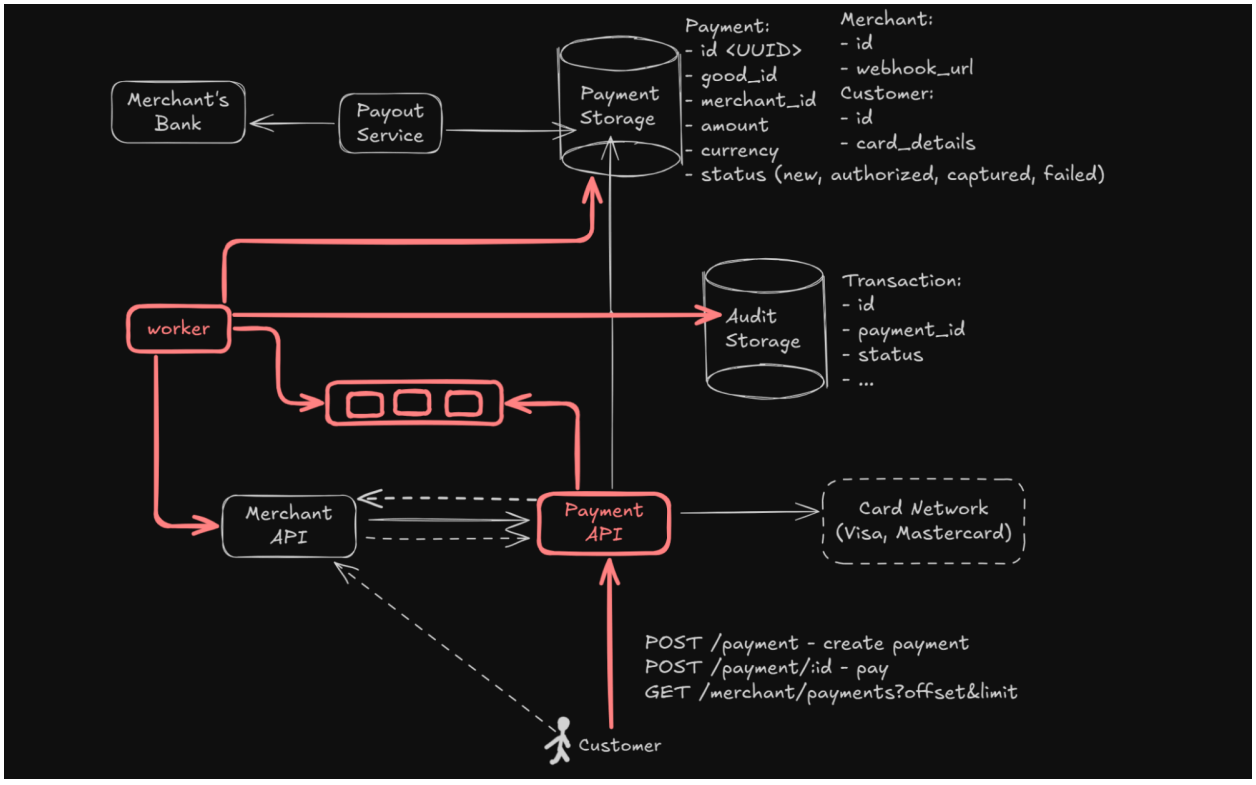
- Apache Kafka
- Pulsar
- Kinesis

In this design we use:

```
None
Kafka-style event queue
```

# 8.10.4 Updated Architecture

## Event-Driven Side Effect Processing




# 8.10.5 New Processing Flow

The flow now becomes:

## Step 1 — Payment API Processes Payment

Payment API:

- Authorizes payment
- Updates Payment Storage

## Step 2 — Publish Payment Event

Instead of directly executing side effects:

Payment API publishes event to Kafka:

Example:

```json
{
  "event_type": "payment.captured",
  "payment_id": "pay_123"
}
```

This operation is:

- Fast
- Durable
- Asynchronous

## Step 3 — Independent Workers Consume Events

Dedicated worker groups consume events independently.

Examples:

| **Worker Type** | **Responsibility** |
| --- | --- |
| Webhook Worker | Send merchant webhook |
| Audit Worker | Store ledger event |

| | |
|---|---|
| Notification Worker | Send customer notifications |
| Analytics Worker | Update reporting systems |
| Payout Worker | Trigger payout processing |

# 8.10.6 Why Kafka Helps

Kafka acts as:

> A durable buffer between systems.

This fundamentally changes system behavior.

## Benefits

### Decoupling

Payment API no longer depends on:

- Merchant webhook latency
- Audit DB availability
- Notification service uptime

### Fault Isolation

If webhook system fails:

- Payments still succeed
- Events remain safely stored in Kafka

Workers can retry later.

### Scalability

Each worker type scales independently.

Example:

```
None
More webhook traffic?

→ Add more webhook workers
```

without scaling the entire payment system.

### Retry Support

Workers can:

- Retry failed events
- Pause consumption
- Resume later

without impacting payment APIs.

### Traffic Spike Handling

Kafka naturally absorbs spikes.

Example:

```
None
Black Friday traffic surge
```

Events accumulate temporarily in queues while workers process them gradually.

# 8.10.7 Event Ordering

Payment events must remain ordered per payment.

Correct sequence:

```
None
NEW

→ AUTHORIZED

→ CAPTURED

→ SETTLED
```

Kafka partitions help preserve ordering.

## Partitioning Strategy

Events are:

```
None
Partitioned by payment_id
```

This guarantees:

- Events for same payment stay ordered
- Different payments process in parallel

# 8.10.8 Durable Side Effects

Previously:

```
None
API crash before audit write
```

could lose ledger entries.

Now:

1. API writes payment
2. API publishes durable Kafka event
3. Worker processes audit write later

Even if workers crash:

- Event remains persisted in Kafka

This significantly improves:

```
None
Reliability
```

# 8.10.9 Worker Failure Handling

Workers may fail due to:

- Database outage
- Merchant downtime
- Network errors

Kafka enables:

- Retry consumption
- Offset management
- Replay capability

This is extremely important for:

- Webhooks
- Payouts
- Audit writes

# 8.10.10 Exactly-Once vs At-Least-Once

In distributed systems:

Exactly-once delivery is extremely difficult.

Therefore most payment architectures use:

```
None
At-least-once event delivery
```

combined with:

```
None
Idempotent consumers
```

Meaning:

- Duplicate events are acceptable
- Duplicate side effects must be safely ignored

# 8.10.11 Independent Consumer Groups

Kafka allows multiple consumer groups.

Same event can be processed independently by:

- Audit workers
- Webhook workers
- Analytics workers

without interfering with each other.

This creates:

```
None
Fan-out architecture
```

# 8.10.12 Improved Reliability

With Kafka:

- Side effects become durable
- Temporary failures become recoverable
- Systems become loosely coupled

This dramatically improves:

- Availability
- Scalability
- Operational safety

# 8.10.13 Important Tradeoff

We gain scalability and reliability at the cost of:

```
None
Eventual consistency
```

Example:

- Payment succeeds immediately
- Webhook may arrive seconds later

This is acceptable because:

- Side effects are asynchronous by nature

# 8.10.14 Architectural Insight

Introducing Kafka is one of the most important scalability improvements because it transforms the architecture from:

```
None
Synchronous Request Chain
```

into:

```
Distributed Event-Driven System
```

This becomes the foundation for:

- Durable retries
- Massive scalability
- Failure recovery
- Loose coupling
- Replayability
- Independent service evolution

All modern large-scale payment systems heavily rely on this architectural pattern.

# 8.11 Correctness and Reliable Delivery Guarantees

In a distributed payment system, correctness is far more important than raw performance.

The system must guarantee:

- No money loss
- No missing audit records
- Reliable webhook delivery
- Recoverability after crashes
- Durable retries

To achieve this, the architecture introduces:

> Durable retry storage + at-least-once processing semantics.

## 8.11.1 The Core Problem

Distributed systems fail constantly.

Possible failures include:

- Worker crashes
- Network timeouts
- Merchant API downtime
- Database failures
- Process restarts
- Partial execution failures

Example:

```
None
Webhook successfully sent

but worker crashes before acknowledging completion
```

Now the system does not know:

- Was webhook delivered?
- Should it retry?
- Will duplicate event occur?

# 8.11.2 Why Reliability Is Critical

Two operations are especially sensitive:

| **Operation** | **Why Important** |
| --- | --- |
| Webhook delivery | Merchant business logic depends on it |
| Audit/Ledger writes | Legal compliance requirement |

Failures here cannot silently lose data.

# 8.11.3 Introducing Durable Retry Storage

To solve this, we introduce:

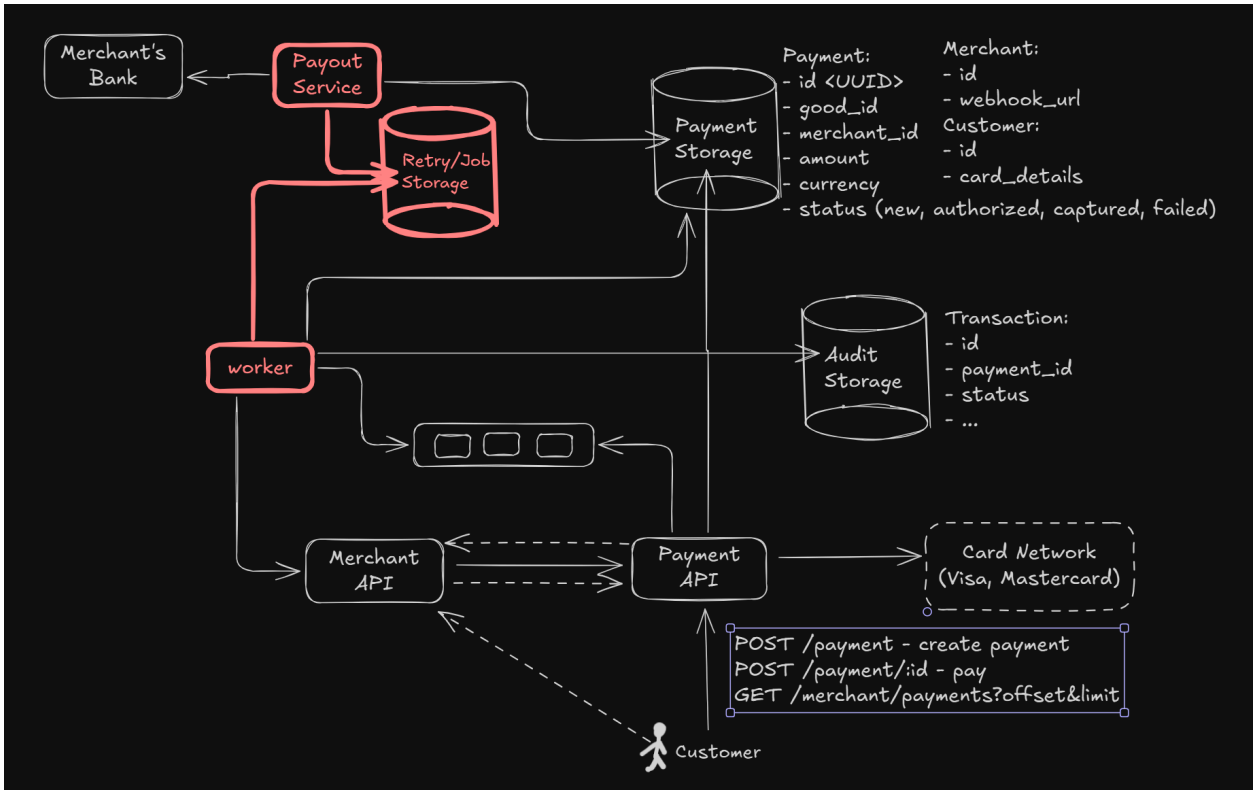
```
None
Retry / Job Storage
```

Purpose:

- Persist unfinished work
- Track retries
- Recover after crashes
- Support long-running retries

# 8.11.4 Updated Reliability Architecture

## Reliable Processing With Durable Retries




# 8.11.5 Reliable Webhook Delivery Flow

The webhook flow now becomes:

## Step 1 — Payment Event Published

Payment API publishes event into Kafka:

Example:

```json
{
  "event_type": "payment.captured",
  "payment_id": "pay_123"
```

```
}
```

## Step 2 — Worker Consumes Event

Webhook worker picks event from queue.

## Step 3 — Persist Retry Job Before Execution

Before calling Merchant API:

- Worker creates durable retry job

Example:

```JSON
{
  "job_id": "job_789",
  "type": "webhook_delivery",
  "status": "PENDING",
  "payment_id": "pay_123",
  "retry_count": 0
}
```

This is critical because:

```None
Work becomes recoverable
```

even if worker crashes immediately afterward.

## Step 4 — Send Webhook

Worker sends:

```
None
POST merchant/webhook
```

## Step 5 — Update Retry Status

If successful:

```
None
PENDING → COMPLETED
```

If failed:

```
None
PENDING → RETRY
```

# 8.11.6 At-Least-Once Delivery Semantics

The system guarantees:

> Webhooks will be delivered at least once.

Meaning:

- Duplicate deliveries are possible
- Silent data loss is unacceptable

This is the standard reliability tradeoff in distributed systems.

## Why Not Exactly-Once?

True exactly-once delivery across distributed systems is:

- Extremely difficult
- Expensive
- Often impossible with external systems

Merchant APIs themselves are:

```
None
Outside our transactional boundary
```

Therefore:

```
None
At-least-once + Idempotency
```

is the practical solution.

# 8.11.7 Merchant Idempotency Requirement

Because duplicate deliveries may happen:

Merchants must process webhooks idempotently.

Typical strategy:

- Store event_id
- Ignore already-processed events

Example:

```
None
Same webhook arrives twice

→ Process only once
```

# 8.11.8 Infinite Retry Philosophy

For critical operations:

- We prioritize durability over speed

Therefore:

```
None
Retries continue indefinitely
```

(or extremely long periods like 30 days).

Why?

> Losing financial events is worse than duplicate retries.

# 8.11.9 Audit Storage Reliability

The exact same reliability model is used for:

```
None
Audit / Ledger writes
```

Why?

- Audit logs are legally required
- Missing ledger entries are catastrophic

## Durable Audit Processing

Flow:

1. Payment event stored in Kafka
2. Audit worker consumes event
3. Retry job persisted

4. Audit record written
5. Completion acknowledged

If worker crashes:

- Retry job survives
- Event can be replayed

# 8.11.10 Recovery After Worker Crash

Suppose:

1. Worker processes webhook
2. Machine crashes midway

Recovery mechanism:

- Retry storage still contains unfinished job
- Another worker picks it up later

This provides:

```
None
Crash recoverability
```

# 8.11.11 Why Retry Storage Matters

Without durable retry tracking:

- In-flight operations disappear on crashes
- Distributed retries become unreliable
- Long outages become impossible to recover from

Retry storage acts as:

> Persistent workflow memory.

# 8.11.12 Scaling Retry Infrastructure

Retry systems themselves must scale.

Typical approaches:

- Shard by job_id
- Leaderless replication
- Delayed retry queues
- Priority scheduling

This allows handling:

- Millions of pending retries
- Long merchant outages
- Massive retry storms

# 8.11.13 Tradeoff: Duplicates vs Data Loss

Distributed systems usually choose between:

| **Choice** | **Consequence** |
| --- | --- |
| Avoid duplicates | Risk losing data |
| Avoid data loss | Risk duplicates |

Payment systems choose:

```
None
Never lose data
```

Therefore duplicates are tolerated and handled through idempotency.

# 8.11.14 Correctness Guarantees Achieved

With Kafka + Retry Storage + Workers:

| **Guarantee** | **Achieved By** |
|---|---|
| Reliable webhook delivery | Durable retries |
| Audit durability | Persistent event processing |
| Crash recovery | Retry/job storage |
| Retry support | Worker orchestration |
| Fault tolerance | Event replay |
| No silent event loss | At-least-once semantics |

# 8.11.15 Important Architectural Insight

The architecture now embraces a fundamental distributed systems principle:

> Durability is more important than avoiding duplicates.

This is why large-scale payment systems rely heavily on:

- Durable queues
- Persistent retries
- Idempotent consumers
- Replayable event logs

These mechanisms together provide:

```
None
Eventually correct distributed processing
```

even under continuous failures and retries.

# 8.12 Scaling the Payment API

The Payment API is the entry point for all customer payment requests.

Its responsibilities include:

- Creating payments
- Processing authorization requests
- Sending capture requests
- Communicating with Card Networks
- Publishing payment events
- Returning payment status

Since payment traffic can reach:

```
None
10,000+ requests/sec at peak
```

the API layer must scale horizontally and remain highly available.

# 8.12.1 Stateless Service Design

The most important architectural property of the Payment API is:

```
None
Statelessness
```

Meaning:

- API servers do not store session state locally
- All persistent data is stored externally

Examples:

- Payment data → Payment Storage
- Events → Kafka
- Retry jobs → Retry Storage
- Merchant info → Merchant Storage

# 8.12.2 Why Statelessness Matters

Because servers hold no local state:

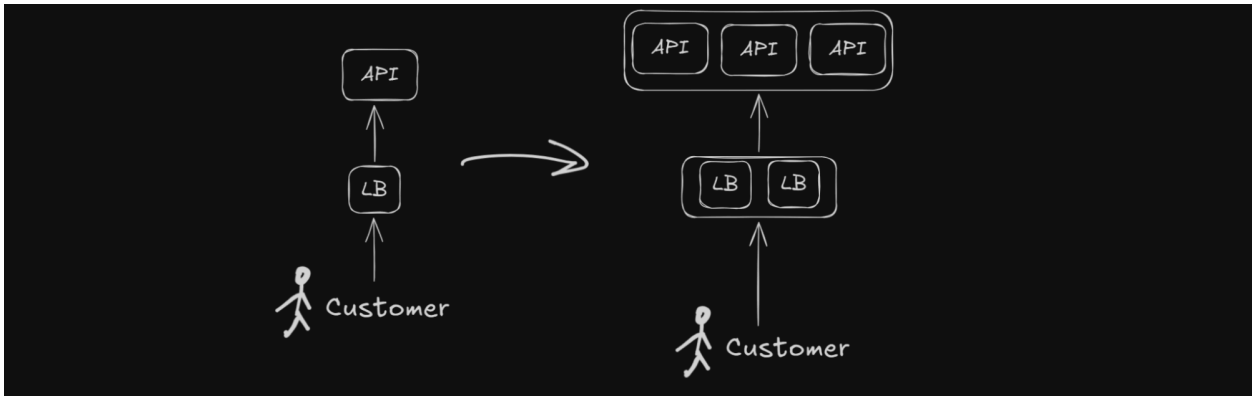
- Any request can go to any API instance
- Failed servers can be replaced instantly
- Horizontal scaling becomes trivial

This enables:

```
None
Elastic scaling
```

# 8.12.3 Basic Scaling Architecture

## Horizontally Scaled API Layer




# 8.12.4 Traffic Flow

The request path becomes:

```
Customer

→ Load Balancer

→ Payment API Instance

→ Storage / Card Network
```

# 8.12.5 Horizontal Scaling

When traffic increases:

- Add more API servers

Example:

| **Traffic** | **API Instances** |
| --- | --- |

| | |
|---|---|
| 1k rps | 5 instances |
| 10k rps | 50 instances |
| Black Friday | Auto-scale dynamically |

Since servers are stateless:

```
None
No complex coordination required
```

# 8.12.6 Load Balancers

Load balancers distribute incoming requests across API instances.

Responsibilities:

- Traffic distribution
- Health checks
- Failover
- TLS termination
- DDoS protection
- Geographic routing

## Common Load Balancer Types

| **Type** | **Examples** |
|---|---|
| Layer 4 | TCP Load Balancer |

| | |
|---|---|
| Layer 7 | HTTP Reverse Proxy |
| Managed Cloud LB | ALB, NLB, GCLB |

# 8.12.7 Multi-Region Deployment

Payment systems require:

```
None
High availability across regions
```

Therefore APIs are deployed globally.

Example regions:

- US-East
- US-West
- Europe
- Asia

Benefits:

- Lower latency
- Disaster recovery
- Regional failover
- Traffic isolation

# 8.12.8 Active-Active Architecture

Typically payment APIs run in:

```
None
Active-Active mode
```

Meaning:

- Multiple regions serve traffic simultaneously
- If one region fails, traffic shifts automatically

This improves:

- Availability
- Fault tolerance
- Resilience

# 8.12.9 API Failover

Suppose:

- One API node crashes

Load balancer:

- Detects failed health checks
- Stops routing traffic to unhealthy node

No customer impact occurs because:

```
None
Other stateless instances continue serving traffic
```

# 8.12.10 Auto Scaling

Traffic in payment systems is highly uneven.

Examples:

- Black Friday
- Flash sales
- Festival seasons
- Large merchant campaigns

Therefore APIs use:

```
None
Auto-scaling groups
```

Scaling metrics may include:

- CPU usage
- Request rate
- Queue depth
- Latency
- Error rate

# 8.12.11 Connection Pooling

Payment APIs frequently communicate with:

- Databases
- Kafka
- Card Networks

Opening new connections per request is expensive.

Therefore APIs maintain:

```
None
Connection pools
```

Benefits:

- Lower latency
- Reduced overhead
- Better throughput

# 8.12.12 Rate Limiting

To protect the system:

- APIs enforce rate limits

This prevents:

- Abuse
- DDoS attacks
- Merchant bugs
- Retry storms

Typical limits:

- Per API key
- Per merchant
- Per IP

# 8.12.13 Idempotent APIs

Payment APIs must safely handle retries.

Example:

```
None
Customer retries payment request

because of timeout
```

API must avoid duplicate charges.

Therefore endpoints support:

```
None
Idempotency keys
```

Example:

```
None
POST /payment

Idempotency-Key: abc123
```

Repeated requests:

- Return same result
- Do not create duplicate payments

# 8.12.14 Observability

Large-scale API systems require strong observability.

Critical metrics:

- Request latency
- Error rates
- Authorization failures
- Capture success rates
- Region health
- Queue publish failures

Monitoring tools:

- Prometheus
- Grafana
- Datadog
- OpenTelemetry

# 8.12.15 Security Considerations

Since APIs process card payments:

- Security is critical

Important protections:

- TLS everywhere
- API authentication
- PCI DSS compliance
- WAF protection
- Request signing
- Encryption of sensitive data

# 8.12.16 Availability Characteristics

Stateless APIs provide excellent availability because:

- Instances are replaceable
- Scaling is simple
- Failover is fast
- Deployments are safer

This aligns perfectly with payment system goals:

```
None
High availability with horizontal scalability
```

# 8.12.17 Architectural Insight

The API layer is intentionally designed to be:

- Lightweight
- Stateless
- Horizontally scalable
- Easily replaceable

This allows the system to:

- Handle massive traffic spikes
- Survive server failures
- Scale globally
- Maintain low latency

while pushing durability and consistency responsibilities into:

- Storage systems
- Event queues
- Retry infrastructure
- Distributed transaction flows.

# 8.13 Payment Storage Design

The Payment Storage layer is one of the most critical components of the entire payment gateway architecture.

It stores:

- Payment entities
- Authorization state
- Capture state
- Payment lifecycle transitions
- References to card network transactions

This storage is:

```
None
Write-heavy and latency-sensitive
```

because every payment operation updates payment state multiple times.

# 8.13.1 Storage Requirements

The payment database must support:

| **Requirement** | **Importance** |
| --- | --- |

| | |
|---|---|
| Massive write throughput | Critical |
| Strong consistency | Required |
| Low latency updates | Required |
| Horizontal scalability | Required |
| High availability | Required |
| Reliable deduplication | Required |

# 8.13.2 Why Scaling Is Necessary

Expected scale:

- Hundreds of millions of payments/day
- Thousands of writes/sec
- Payment state transitions for every transaction

Single database servers eventually become:

```
None
Write bottlenecks
```

Therefore:

> We horizontally partition the payment data.

# 8.13.3 Sharding the Payment Database

The primary scaling technique is:

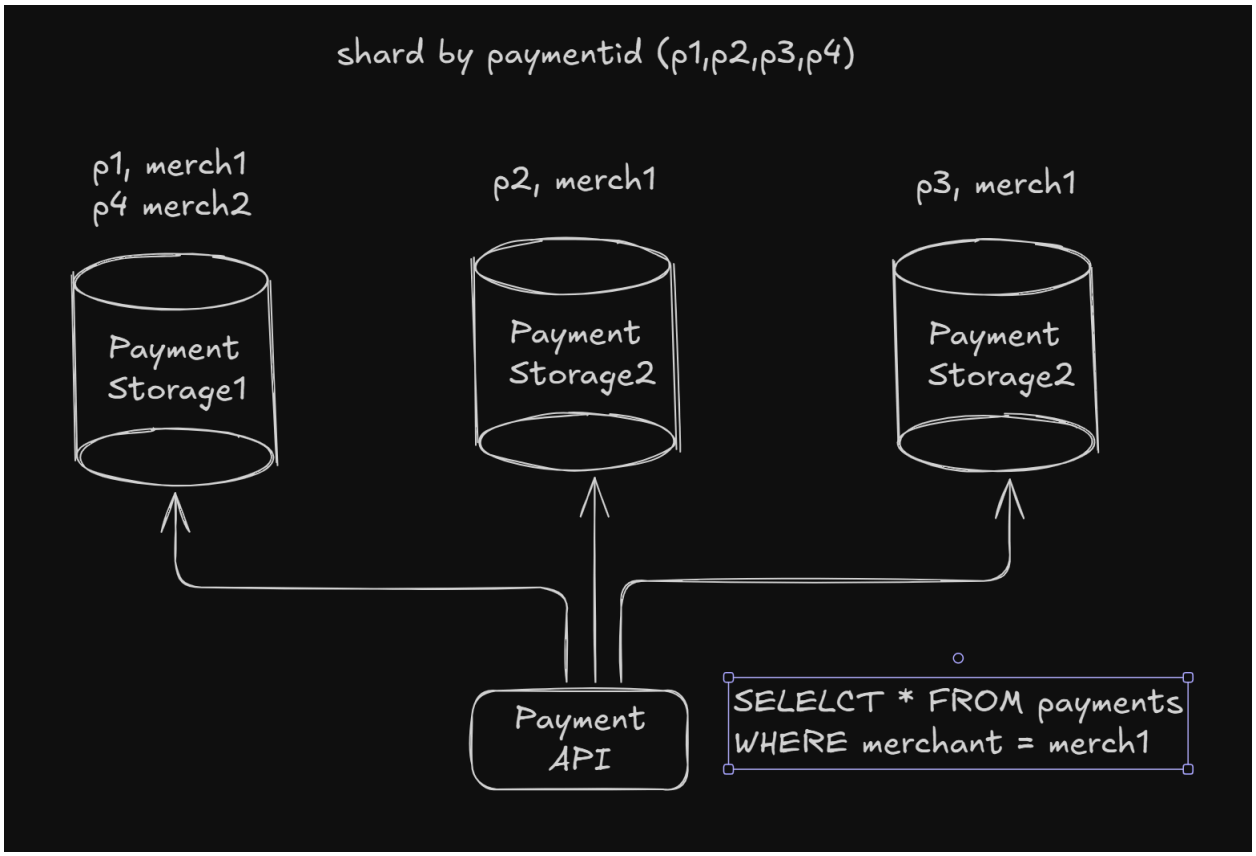
```
None
Database Sharding
```

Meaning:

- Split data across multiple database nodes
- Each node stores only part of the dataset

# 8.13.4 Sharding Strategy

## Shard by Payment ID



# 8.13.5 Why Shard by Payment ID?

We shard using:

```
None
payment_id
```

instead of:

```
None
merchant_id
```

because merchant-based sharding creates severe hotspot risks.

## Example Hotspot Problem

Suppose:

```
None
Amazon-like merchant
```

receives enormous traffic.

If sharded by merchant_id:

- One shard receives huge traffic
- Other shards remain underutilized

This creates:

- Uneven load
- Hot partitions
- Scalability collapse

# 8.13.6 Benefits of Payment-ID Sharding

Payment IDs are:

- Random
- Uniformly distributed
- Independent across merchants

This provides:

```
None
Even write distribution
```

across shards.

Benefits:

- Better scalability
- Better throughput
- Reduced hotspots
- Predictable balancing

# 8.13.7 Consistent / Rendezvous Hashing

To map payments to shards we use:

- Consistent hashing
  or
- Rendezvous hashing

These techniques help:

- Add/remove shards dynamically
- Minimize data movement
- Simplify rebalancing

## Why Normal Hashing Is Bad

With simple modulo hashing:

```
None
hash(payment_id) % N
```

adding a new shard changes:

```
None
Almost all mappings
```

causing massive data reshuffling.

# 8.13.8 Consistent Hashing Benefits

Consistent hashing minimizes:

- Data migration
- Rebalancing overhead
- Cache invalidation

Only a small subset of keys move when:

- New shards added
- Shards removed
- Failures occur

# 8.13.9 Strong Consistency Requirements

Payments require:

```
None
Linearizable state transitions
```

because duplicate charges are unacceptable.

Examples:

- `NEW → AUTHORIZING`
- `AUTHORIZING → AUTHORIZED`
- `AUTHORIZED → CAPTURED`

must be strongly consistent.

# 8.13.10 CAS-Based State Transitions

To prevent double charging:

- Storage uses Compare-And-Swap (CAS)

Example:

```sql
UPDATE payments

SET status = 'authorizing'

WHERE id = $id

AND status = 'new';
```

Only one request succeeds.

This guarantees:

```
Single successful authorization flow
```

# 8.13.11 The Merchant Dashboard Problem

Sharding by payment_id creates a new challenge.

Suppose merchant dashboard executes:

```sql
SELECT * FROM payments

WHERE merchant_id = 'merch1';
```

Payments for the same merchant may exist on:

- Shard 1
- Shard 2
- Shard 3
- Shard N

This creates:

```
Scatter-Gather Queries
```

# 8.13.12 What Is Scatter-Gather?

Coordinator must:

1. Query every shard
2. Aggregate results
3. Merge responses
4. Return final dataset

Problems:

- Expensive
- Slow
- Poor scalability
- High fan-out load

At Stripe-scale this becomes:

```
A major bottleneck
```

# 8.13.13 Derived Storage Pattern

To solve this:

> We introduce another storage optimized for merchant queries.

Called:

```
None
Merchant Storage
```

# 8.13.14 Merchant Storage Purpose

Merchant Storage contains:

- Merchant metadata
- Merchant dashboard data
- Denormalized payment records
- Query-optimized views

Unlike Payment Storage:

```
None
It is optimized for reads and aggregations
```

# 8.13.15 Merchant Storage Sharding

Merchant Storage is:

```
None
Sharded by merchant_id
```

This enables:

- Efficient dashboard queries

- Fast aggregations
- Pagination
- Reporting

# 8.13.16 Data Synchronization

Payment data is asynchronously copied from:

```
None
Payment Storage

→ Merchant Storage
```

Typically via:

- Kafka events
- CDC streams
- Background workers

This creates:

```
None
Derived / materialized views
```

# 8.13.17 Tradeoff: Eventual Consistency

Merchant Storage becomes:

```
None
Eventually consistent
```

Meaning:

- Dashboard may lag slightly

- Recent payments may appear after a delay

This is acceptable because:

- Merchant dashboards are not transactional systems

# 8.13.18 Separation of Concerns

The architecture now separates workloads:

| **Storage** | **Optimized For** |
|---|---|
| Payment Storage | Writes + strong consistency |
| Merchant Storage | Reads + aggregations |

This is a very important scalability principle.

# 8.13.19 Replication Strategy

Payment Storage typically uses:

- Leader-follower replication
  or
- Consensus-based replication

to ensure:

- Durability
- High availability
- Failover support

Replication is especially important because:

```
None
Payment data cannot be lost
```

# 8.13.20 Architectural Insight

This design follows a core distributed systems principle:

> Different workloads require different storage models.

Instead of forcing one database to handle:

- transactional writes,
- dashboard reads,
- analytics,
- aggregations,

we split responsibilities into:

- transactional storage,
- derived storage,
- analytical systems.

This dramatically improves:

- scalability,
- throughput,
- query performance,
- and operational stability.

# 8.14 Merchant Storage Design

As discussed in the previous section, the Payment Storage layer is optimized for:

- high write throughput,
- strong consistency,
- and even distribution of payment records.

However, this creates problems for:

```
None
Merchant-facing queries
```

such as:

- Merchant dashboards
- Payment history
- Analytics
- Reporting
- Search/filtering

To solve this, we introduce a dedicated:

```
None
Merchant Storage
```

optimized specifically for merchant access patterns.

# 8.14.1 Purpose of Merchant Storage

Merchant Storage acts as:

> A derived, query-optimized storage layer.

It stores:

- Merchant metadata
- Merchant payment views
- Denormalized payment records
- Dashboard-related entities

Unlike Payment Storage:

```
None
It prioritizes read efficiency over transactional writes
```

# 8.14.2 Why Separate Merchant Storage?

If merchants directly queried Payment Storage:

```sql
SELECT * FROM payments

WHERE merchant_id = 'merchant_1';
```

the system would need:

- scatter-gather queries across shards,
- fan-out requests,
- result aggregation,
- distributed sorting/pagination.

At large scale this becomes:

```
Operationally expensive and slow
```

# 8.14.3 Dedicated Merchant-Oriented Storage

Instead:

- payment data is copied asynchronously
- into a storage optimized for merchant queries

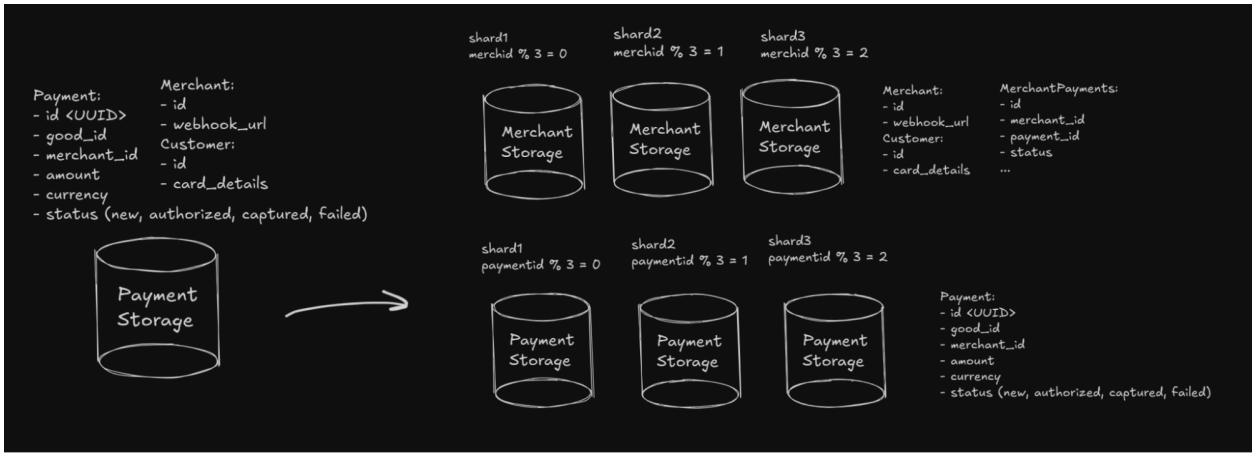
This creates:

```
Materialized / derived views
```

for merchants.

# 8.14.4 Merchant Storage Architecture

## Merchant-Centric Sharding




# 8.14.5 Sharding Strategy

Merchant Storage is:

```
None
Sharded by merchant_id
```

because merchant-related queries naturally group around:

- a single merchant,
- a merchant dashboard,
- or merchant analytics.

# 8.14.6 Why Merchant-ID Sharding Works Here

Merchant queries usually require:

| Query Type | Example |
| --- | --- |
| Merchant payments | All payments for merchant |
| Payment history | Recent transactions |
| Revenue analytics | Sum/group by date |
| Failed payments | Filter by status |

All of these become efficient when:

```
None
Merchant data lives on the same shard
```

# 8.14.7 Data Model

Typical entities stored:

## Merchant Entity

```
JSON
{
  "merchant_id": "m123",
  "webhook_url": "https://merchant.com/webhook"
```

```
}
```

### MerchantPayments Entity

```JSON
{
  "payment_id": "p789",
  "merchant_id": "m123",
  "status": "captured",
  "amount": 100
}
```

This creates:

```None
Denormalized query-friendly storage
```

# 8.14.8 Data Synchronization Flow

Merchant Storage is populated asynchronously from:

```None
Payment Events
```

Flow:

```
None
Payment API

→ Kafka Event

→ Worker

→ Merchant Storage
```

Workers continuously:

- consume payment events,
- transform data,
- update merchant views.

# 8.14.9 Eventual Consistency

Because synchronization is asynchronous:

```
None
Merchant Storage is eventually consistent
```

Meaning:

- Dashboard updates may lag slightly
- Recent payments may appear with delay

This tradeoff is acceptable because:

- Merchant dashboards are not part of payment correctness

# 8.14.10 Consistent / Rendezvous Hashing

To distribute merchants across shards:

- Consistent hashing
  or
- Rendezvous hashing

is used.

Benefits:

- Easy shard addition/removal
- Minimal rebalancing
- Better scalability

# 8.14.11 The Celebrity Problem

One major challenge remains:

Some merchants are:

```
None
Extremely large traffic hotspots
```

Examples:

- Amazon
- Uber
- Netflix
- Shopify-scale merchants

These merchants can overwhelm a shard.

This is known as:

```
None
The Celebrity Problem
```

# 8.14.12 Why Celebrity Merchants Are Dangerous

Suppose:

```
None
merchant_id % N
```

places a huge merchant onto one shard.

That shard may experience:

- Excessive writes
- Query overload
- Hot partitions
- High replication lag

while other shards remain mostly idle.

# 8.14.13 Handling Hot Merchants

Large merchants are often:

```
None
Isolated onto dedicated infrastructure
```

Strategies include:

- Dedicated shards
- Vertical scaling
- Merchant-specific partitions
- Multi-level sharding

## Example Strategy

Instead of:

```
None
Shard by merchant_id only
```

use:

```
None
merchant_id + payment_id
```

for extremely large merchants.

# 8.14.14 Read Optimization

Merchant Storage may additionally support:

- Secondary indexes
- Time-based partitioning
- OLAP systems
- Search indexes

to improve:

- reporting,
- analytics,
- dashboard filtering,
- exports.

# 8.14.15 Replication and Availability

Merchant Storage typically prioritizes:

```
None
Availability over strict consistency
```

because:

- dashboards tolerate temporary staleness,
- payment correctness does not depend on this storage.

Therefore:

- leader-follower replication,
- quorum reads,
- or eventually consistent replication

are acceptable.

# 8.14.16 Separation of Workloads

The architecture now cleanly separates:

| **Storage** | **Responsibility** |
| --- | --- |
| Payment Storage | Transaction processing |
| Merchant Storage | Merchant queries |
| Audit Storage | Immutable ledger |
| Analytics Systems | Heavy aggregations |

This prevents:

```
None
Mixed workload interference
```

# 8.14.17 Architectural Insight

Merchant Storage is an example of:

```
None
CQRS-style architecture
```

(Command Query Responsibility Segregation)

Where:

- transactional systems optimize writes,
- query systems optimize reads.

This separation is essential at Stripe-scale because:

- write patterns,
- read patterns,
- consistency needs,
- and scalability requirements

are fundamentally different.

# 8.15 Audit Storage

As discussed earlier, Audit Storage acts as the immutable ledger of the payment system. Every important payment event is written here:

- payment created
- authorization started
- authorization completed
- capture completed
- payout initiated
- payout completed
- webhook delivery attempts

This storage is critical for:

- financial reconciliation
- compliance

- debugging
- dispute resolution
- recovery after failures

Unlike Payment Storage, the access patterns for Audit Storage are not clearly defined in advance, which makes choosing a universal sharding strategy difficult.

# 8.15.1 Sharding Strategy

Since we do not know all future query patterns, we use a simple and scalable approach:

```
None
Shard by payment_id
```

same as Payment Storage.

Benefits:

- even distribution of traffic
- natural scalability
- avoids hotspots
- simple routing logic
- colocates payment-related events

Because payment ids are UUIDs, writes distribute uniformly across shards.

# 8.15.2 Why Payment-ID Sharding Works

Audit events naturally belong to a specific payment flow.

Example:

| **payment_id** | **event** |
| --- | --- |

| | |
|---|---|
| p1 | payment_created |
| p1 | authorization_started |
| p1 | authorized |
| p1 | captured |

Keeping all events for the same payment on the same shard improves:

- debugging
- replay operations
- reconciliation
- event tracing

This also aligns well with Payment Storage because both systems share the same partitioning model.

# 8.15.3 Immutable Ledger Design

Audit Storage should behave like an append-only ledger.

Properties:

- entries are never deleted
- historical records are preserved
- writes are immutable
- retries may create duplicates but never data loss

This is important because:

```
None
Losing financial records is worse than duplicate events
```

Workers and consumers should therefore be idempotent.

# 8.15.4 Replication and Durability

Audit data is extremely sensitive.

We cannot afford data loss.

Typical replication strategies:

- leader-follower replication
- leaderless quorum replication
- multi-region replication

Durability techniques:

- synchronous replication
- WAL (write-ahead logging)
- quorum acknowledgements

This ensures audit entries survive:

- node failures
- crashes
- network partitions
- regional outages

# 8.15.5 Asynchronous Writes Through Kafka

Audit writes are performed asynchronously through Kafka/event queues.

Flow:

```
None
Payment API
```

```
→ Kafka Event

→ Worker

→ Audit Storage
```

Benefits:

- decouples Payment API from side effects
- retry support
- replay capability
- at least once delivery guarantees

If a worker crashes before writing to Audit Storage, Kafka retains the event and another worker can retry later.

# 8.15.6 Event Delivery Guarantees

Audit systems usually prioritize:

```
None
At least once delivery
```

instead of exactly-once semantics because:

- duplicates can be handled
- lost financial records cannot be tolerated

Workers therefore continuously retry failed writes until success.

# 8.15.7 Limitations of Payment-ID Sharding

One limitation is analytical workloads.

Queries like:

```sql
SELECT *

FROM audit_logs

WHERE created_at BETWEEN t1 AND t2
```

or

```sql
SELECT SUM(amount)

FROM audit_logs

WHERE status = 'captured'
```

require scatter-gather execution across multiple shards.

This becomes expensive at very large scale.

# 8.15.8 Advanced Analytical Systems

For heavy analytical queries, companies usually introduce dedicated OLAP systems such as:

- ClickHouse
- BigQuery
- Snowflake
- Druid

or streaming systems:

- Kafka Streams
- Flink
- Spark Streaming

These systems are optimized for:

- aggregations
- range scans

- reporting
- fraud analysis
- compliance analytics

# 8.15.9 Future Improvements

As scale grows, more advanced partitioning may be introduced:

- time-based partitioning
- hybrid sharding
- archival tiers
- cold storage

Example:

```
None
payment_id + timestamp
```

This improves:

- retention management
- historical scans
- analytical performance

# 8.15.10 Architectural Insight

Audit Storage represents the system of record for financial history.

The architecture now separates responsibilities cleanly:

| **Storage** | **Responsibility** |
| --- | --- |
| Payment Storage | Transaction processing |

| | |
|---|---|
| Merchant Storage | Merchant queries |
| Audit Storage | Immutable financial ledger |
| Analytics Systems | Heavy aggregations/reporting |

This separation allows the system to independently optimize:

- correctness
- scalability
- durability
- query performance
- operational reliability

# 8.16 PaymentEvents Queue

As discussed earlier, Kafka/message broker is introduced to decouple side effects from the critical payment flow.

The queue is responsible for:

- webhook delivery events
- audit/ledger events
- payout triggers
- notifications
- derived storage synchronization
- retry processing

Since payment systems require ordering within the same payment lifecycle, Kafka topics are partitioned by:

```
None
payment_id
```

This guarantees that all events related to a specific payment are processed in order.

Example:

```
NEW

→ AUTHORIZED

→ CAPTURED

→ SETTLED
```

Events for the same payment always land on the same Kafka partition, preserving ordering guarantees.

At the same time, different payments are distributed across multiple partitions, enabling massive parallelism and scalability.

# 8.16.1 Why Partition by Payment ID?

Partitioning by payment id gives several important guarantees:

- ordered event processing
- simpler state machines
- easier deduplication
- no concurrent conflicting updates

Without ordering guarantees, workers may process:

```
CAPTURED before AUTHORIZED
```

which would corrupt payment state.

# 8.16.2 Kafka Scalability

Kafka scales horizontally through:

```
None
Topic partitions
```

As traffic grows:

- add more brokers
- add more partitions
- rebalance consumers

This allows Kafka to support:

- millions of events/sec
- durable replay
- independent consumers
- fault tolerance

# 8.16.3 Durability and Replay

Kafka also acts as a durable event log.

Benefits:

- replay failed events
- recover crashed workers
- rebuild derived storage
- recover from operational incidents

This is extremely important for:

- audit systems
- reconciliation
- retry processing
- disaster recovery

# 8.17 Workers / Background Jobs

Workers consume events from Kafka and process side effects asynchronously.

Typical workers:

| Worker Type | Responsibility |
|---|---|
| Webhook Worker | Send merchant webhooks |
| Audit Worker | Write ledger entries |
| Payout Worker | Execute payouts |
| Analytics Worker | Update reporting systems |
| Notification Worker | Send emails/SMS |

Workers are intentionally designed as:

```
None
Stateless compute nodes
```

meaning:

- no local durable state
- easy replacement
- trivial horizontal scaling

# 8.17.1 Worker Scalability

Workers scale together with Kafka partitions.

Example:

| Kafka Partitions | Worker Instances |
| --- | --- |
| 10 | 10 workers |
| 100 | 100 workers |
| 1000 | 1000 workers |

Each worker consumes a subset of partitions.

As more partitions are added:

- more workers can be added
- throughput increases linearly

This creates a naturally scalable event-processing architecture.

# 8.17.2 Failure Recovery

If a worker crashes:

- Kafka retains offsets/events
- another worker takes ownership
- processing resumes automatically

This provides:

```
None
Automatic failure recovery
```

without impacting Payment APIs.

# 8.17.3 Idempotent Processing

Because Kafka typically guarantees:

```
None
At least once delivery
```

workers must be idempotent.

Meaning:

- duplicate events are possible
- processing same event twice should be safe

This is critical for:

- payouts
- webhook delivery
- ledger writes

# 8.18 Retry / Job Storage

Some workflows cannot complete immediately.

Examples:

- merchant webhook endpoint down
- payout bank unavailable
- temporary network failure
- third-party API timeout

Therefore we introduce:

```
None
Retry / Job Storage
```

which stores durable background jobs and retry metadata.

# 8.18.1 Sharding Strategy

Retry/Job Storage can be trivially sharded by:

```
None
job_id
```

Benefits:

- even distribution
- simple routing
- horizontal scalability

Since jobs are independent, this partitioning strategy works well.

# 8.18.2 Querying by Job Type

One challenge is querying jobs by:

- webhook jobs
- payout jobs
- reconciliation jobs

Example query:

```sql
SELECT *

FROM jobs

WHERE type = 'webhook_retry'
```

Since jobs are distributed across shards, this may require querying multiple nodes.

To optimize this:

```
None
Indexes are added on job type
```

allowing efficient filtering and scheduling.

# 8.18.3 Retry Semantics

Retry systems usually implement:

- exponential backoff
- delayed retries
- dead letter queues
- retry counters

Critical jobs may retry for:

```
None
Days or even indefinitely
```

because financial correctness is more important than temporary duplication.

# 8.18.4 Durable Workflow Recovery

Retry Storage acts as:

```
None
Persistent workflow memory
```

If workers crash midway:

- unfinished jobs remain stored
- another worker can resume processing later

This is critical for:

- webhook guarantees
- payout correctness
- reconciliation workflows

# 8.19 CAP: Availability vs Consistency

A payment gateway is a distributed system, therefore CAP tradeoffs are unavoidable.

For Stripe-like systems:

```
None
High availability is extremely important
```

because downtime directly impacts merchant revenue.

The system should continue operating even during:

- node failures
- region failures
- network partitions
- traffic spikes

# 8.19.1 Different Components Need Different Guarantees

Not all components require the same consistency level.

## Strong Consistency Required

Critical transactional flows require:

```
None
Linearizable consistency
```

Examples:

- payment authorization
- capture
- deduplication
- balance updates

Why?

```
None
Double charging is unacceptable
```

## Eventual Consistency Acceptable

Other systems can tolerate temporary staleness:

- merchant dashboards
- analytics
- reporting
- webhook status
- derived storage

These systems prioritize:

```
None
Availability and scalability
```

over strict consistency.

# 8.19.2 Availability Strategy

To maximize uptime, components use:

- replication
- load balancing
- multi-region deployment
- automatic failover
- stateless services
- durable queues

This ensures:

```
None
No single machine failure takes down the system
```

# 8.19.3 Eventual Reconciliation

Because distributed systems are asynchronous, temporary inconsistencies are inevitable.

Therefore payment systems rely heavily on:

```
None
Reconciliation processes
```

to eventually restore correctness between:

- banks
- ledgers
- payouts
- internal storage

# 8.19.4 Architectural Insight

The final architecture intentionally combines:

| **Requirement** | **Strategy** |
| --- | --- |

| | |
|---|---|
| Payment correctness | Strong consistency |
| High scalability | Partitioning/sharding |
| Fault tolerance | Replication + queues |
| High availability | Stateless services |
| Reliability | Durable retries |
| Operational recovery | Replay + reconciliation |

This balance between:

- consistency,
- availability,
- and scalability

is what enables modern payment gateways to operate reliably at global scale.

# 8.20 Payment Storage Replication

Payment Storage is the most critical component of the system because it stores:

- payment state
- authorization status
- capture information
- idempotency state
- reconciliation metadata

Unlike dashboards or analytics systems:

```
None
Payment correctness is mission critical
```

Even small inconsistencies can lead to:

- double charges
- lost payments
- duplicate captures
- incorrect balances
- reconciliation failures

Therefore the replication strategy for Payment Storage must prioritize:

```
None
Strong consistency and durability
```

# 8.20.1 Why Simple Leader-Follower Replication Is Not Enough

Traditional:

```
None
Leader-Follower replication
```

is insufficient for payment correctness.

Why?

Followers may lag behind the leader because replication is asynchronous.

This creates stale reads.

Example:

```
None
Payment authorized on leader
```

```
Follower still shows "new"
```

A client reading stale data may:

- retry authorization,
- issue duplicate operations,
- or send unnecessary requests to the Card Network.

In payment systems this is unacceptable.

# 8.20.2 Linearizability Requirement

Payment Storage requires:

```
None
Linearizable consistency
```

Meaning:

Every read must observe the latest committed write.

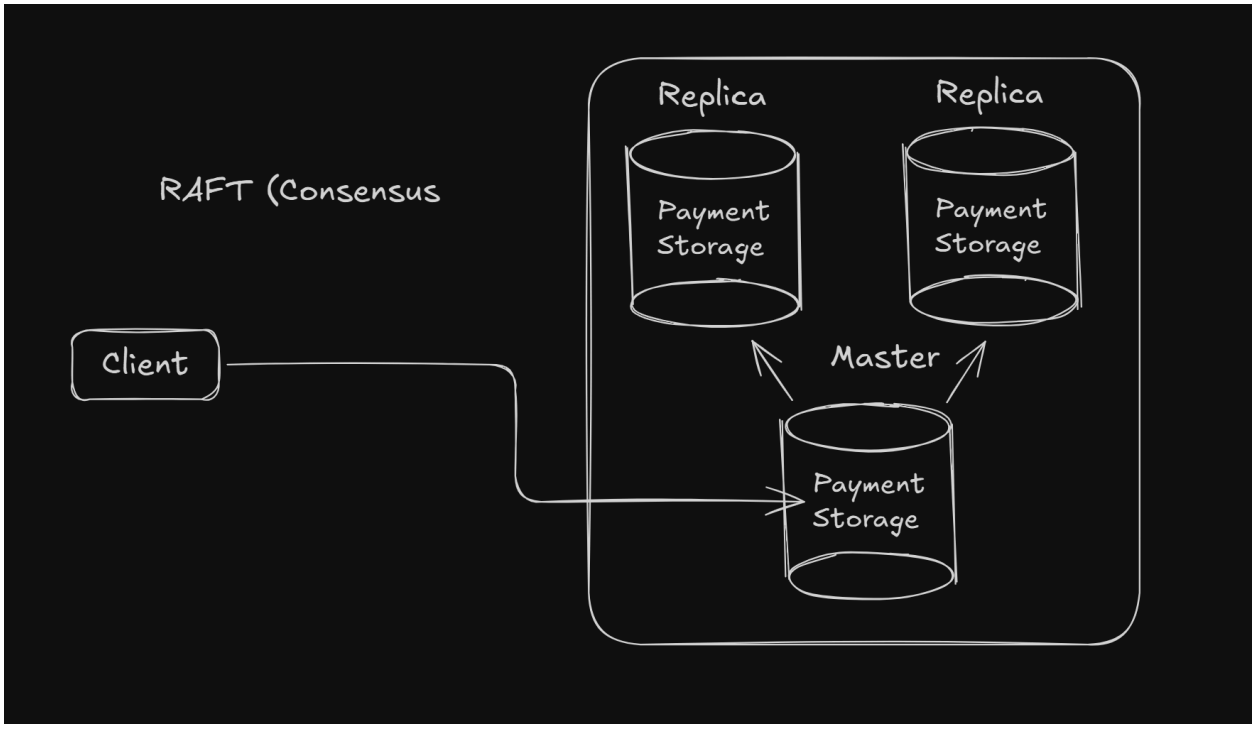
This guarantees:

- no stale payment state
- safe idempotency checks
- correct compare-and-swap logic
- accurate authorization status

Without recency guarantees:

- duplicate payment attempts become possible,
- retries become dangerous,
- concurrency control breaks down.

# 8.20.3 Consensus-Based Replication




To achieve strong consistency, Payment Storage uses:

```
None
Consensus-based replication
```

such as:

- Raft
- Paxos

These protocols ensure:

- replicas agree on ordering,
- writes are committed consistently,
- leader election is safe,
- failover preserves correctness.

# 8.20.4 Raft Replication Model

In a Raft cluster:

- one node acts as leader,
- others act as replicas/followers.

Flow:

```
None
Client

→ Leader

→ Replicated to quorum

→ Commit acknowledged
```

A write succeeds only after a majority of replicas confirm persistence.

This guarantees:

```
None
No committed payment data is lost
```

even during failures.

# 8.20.5 Why Consensus Matters for Payments

Payment systems depend heavily on:

- compare-and-swap updates,
- idempotency,
- sequencing,
- deduplication.

Example:

```SQL
UPDATE payments

SET status = 'authorizing'

WHERE id = $id

AND status = 'new'
```

This logic assumes:

```None
All nodes agree on the latest state
```

Otherwise:

- multiple clients may authorize simultaneously,
- causing duplicate charges.

Consensus replication prevents this.

# 8.20.6 Failure Handling

Suppose the leader crashes.

Raft automatically:

- elects a new leader,
- preserves committed logs,
- resumes operations safely.

Because writes require quorum acknowledgment:

```None
Committed payment records survive node failures
```

This provides both:

- durability
- high availability

# 8.20.7 Tradeoff: Latency vs Correctness

Consensus systems are slower than asynchronous replication because every write requires:

- network coordination
- quorum acknowledgment
- distributed agreement

This increases:

- write latency
- operational complexity

However:

```
None
Correctness is more important than low latency
```

for payment processing systems.

A few extra milliseconds are acceptable.

Incorrect financial transactions are not.

# 8.20.8 Multi-Region Replication

At Stripe scale, Payment Storage is typically replicated across:

- multiple availability zones
- or multiple regions

Benefits:

- disaster recovery
- zone failure tolerance
- regional failover
- improved durability

However global consensus introduces higher latency.

Therefore systems often use:

```
None
Regional consensus clusters
```

instead of a single global cluster.

# 8.20.9 Sharding + Consensus

Earlier we introduced:

```
None
Sharding by payment_id
```

Each shard itself can internally run a:

- Raft cluster
- or Paxos group

Result:

| **Layer** | **Responsibility** |
| --- | --- |
| Sharding | Horizontal scalability |
| Consensus | Strong consistency |

This combination enables:

- massive scale,
- fault tolerance,
- and payment correctness simultaneously.

# 8.20.10 Architectural Insight

Payment Storage is one of the few systems where:

```
None
Consistency is prioritized over availability
```

during network partitions.

Because:

- losing availability temporarily is acceptable,
- corrupting financial state is not.

This is a classic:

```
None
CP-oriented distributed system
```

under the CAP theorem.

The system intentionally sacrifices some availability to guarantee:

- correctness,
- durability,
- and linearizable payment state.

# 8.21 Merchant Storage Replication

Unlike Payment Storage, Merchant Storage is not part of the critical payment execution path.

Its primary responsibility is:

- merchant dashboards
- payment history
- analytics
- reporting
- search/filtering

Therefore:

```
None
Strict consistency is not required
```

If a payment appears slightly later in the merchant dashboard, the actual payment correctness is unaffected.

# 8.21.1 Eventual Consistency Model

Merchant Storage operates as:

```
None
Eventually consistent storage
```

because payment data is synchronized asynchronously from payment events.

Flow:

```
None
Payment API

→ Kafka Event

→ Worker

→ Merchant Storage
```

As a result:

- dashboard data may lag,
- reports may update later,
- analytics may not reflect the latest transaction immediately.

Typical delay tolerance:

```
None
Seconds to even hours
```

depending on backlog or operational conditions.

## 8.21.2 Why Eventual Consistency Is Acceptable

Merchant dashboards are:

```
None
Operational convenience systems
```

not correctness-critical systems.

Even if the dashboard temporarily shows stale data:

- customers are still charged correctly,
- money movement still succeeds,
- ledger correctness is preserved.

The real source of truth remains:

```
None
Payment Storage + Audit Storage
```

not Merchant Storage.

## 8.21.3 Replication Strategy

Because availability is prioritized over strict consistency, the natural replication strategy is:

```
None
Leader-Follower asynchronous replication
```

Benefits:

- low write latency
- high read scalability
- simple architecture
- easy failover
- operational simplicity

Reads can be distributed across replicas to support:

- dashboards
- analytics
- merchant APIs

# 8.21.4 Availability Prioritized Over Consistency

Merchant systems should remain available even during:

- replica lag
- partial failures
- regional issues

Therefore the system intentionally tolerates:

```
None
Temporary stale reads
```

This aligns with an:

```
None
AP-oriented system design
```

under the CAP theorem.

# 8.21.5 Possible Data Loss

Asynchronous replication introduces a risk:

```
Replica data loss during failures
```

Example:

- leader acknowledges write,
- replication has not completed,
- leader crashes permanently.

Recently written merchant data may disappear.

# 8.21.6 Is Data Loss Acceptable?

This becomes a product/business decision.

One possible argument:

```
Merchant dashboard data is reconstructable
```

because:

- the canonical payment record still exists,
- audit logs remain durable,
- Kafka events can be replayed.

Therefore losing temporary derived merchant views may be acceptable.

# 8.21.7 Stronger Replication Alternatives

If product requirements demand stronger guarantees, Merchant Storage can adopt:

- consensus-based replication (Raft/Paxos)
- leader-leader replication
- leaderless/quorum systems

These approaches reduce risk of data loss but introduce:

- higher latency
- increased operational complexity
- harder conflict resolution

# 8.21.8 Rebuilding Merchant Storage

An important advantage of event-driven architecture is:

```
None
Derived storage can be rebuilt
```

Since Merchant Storage is populated from Kafka/payment events:

- replay event streams,
- rebuild merchant views,
- restore missing data,
- regenerate dashboards.

This makes Merchant Storage much more tolerant to failures compared to Payment Storage.

# 8.21.9 Separation of Source of Truth

The architecture intentionally separates:

| **Storage** | **Role** |
| --- | --- |
| Payment Storage | Transaction correctness |

| | |
|---|---|
| Audit Storage | Immutable financial ledger |
| Merchant Storage | Query/read optimization |

This distinction is critical.

Even if Merchant Storage experiences:

- lag,
- temporary inconsistency,
- or partial data loss,

the actual financial system remains correct because:

```
None
Payments are validated against Audit Storage
```

not against dashboard views.

# 8.21.10 Architectural Insight

Merchant Storage demonstrates an important distributed systems principle:

```
None
Not every system requires strong consistency
```

By relaxing consistency requirements for non-critical workloads, the system gains:

- higher scalability
- better availability
- lower latency
- simpler operations

while preserving strong guarantees only where they truly matter:

- payments,

- balances,
- and financial correctness.

# 8.22 Audit Storage Replication

Audit Storage is one of the most critical components of the system because it acts as:

- immutable financial ledger
- compliance storage
- reconciliation source
- historical transaction archive

Every payment update generates an audit entry.

Example:

```
None
NEW

→ AUTHORIZING

→ AUTHORIZED

→ CAPTURED

→ REFUNDED
```

These records are essential for:

- legal compliance,
- financial audits,
- reconciliation,
- dispute resolution,
- regulatory investigations.

# 8.22.1 Data Loss Is Unacceptable

Unlike Merchant Storage:

```
None
Audit data cannot be lost
```

because:

- financial regulations require durability,
- missing ledger records violate compliance,
- reconciliation becomes impossible.

Therefore simple:

```
None
Leader-Follower async replication
```

is insufficient.

Why?

If the leader crashes before replication completes:

- committed audit entries may disappear permanently.

This is unacceptable for financial systems.

# 8.22.2 Linearizability Is NOT Required

Although durability is critical, Audit Storage does not require:

```
None
Linearizable consistency
```

because the system does not:

- make transactional decisions from audit data,
- perform compare-and-swap updates,
- use audit records for payment authorization.

Audit entries are primarily:

- append-only,
- asynchronously written,
- and eventually consumed later.

Therefore temporary replica inconsistency is acceptable.

## 8.22.3 Eventual Consistency Model

Audit Storage behaves as:

```
None
Eventually consistent immutable storage
```

The payment flow continues independently while audit entries propagate across replicas asynchronously.

This creates a very different consistency requirement compared to Payment Storage.

## 8.22.4 Why Consensus Replication Is Unnecessary

Consensus systems like:

- Raft
- Paxos

provide:

- linearizability,
- strict ordering,
- quorum coordination.

However Audit Storage does not require:

- strict recency guarantees,

- synchronized reads,
- transactional coordination.

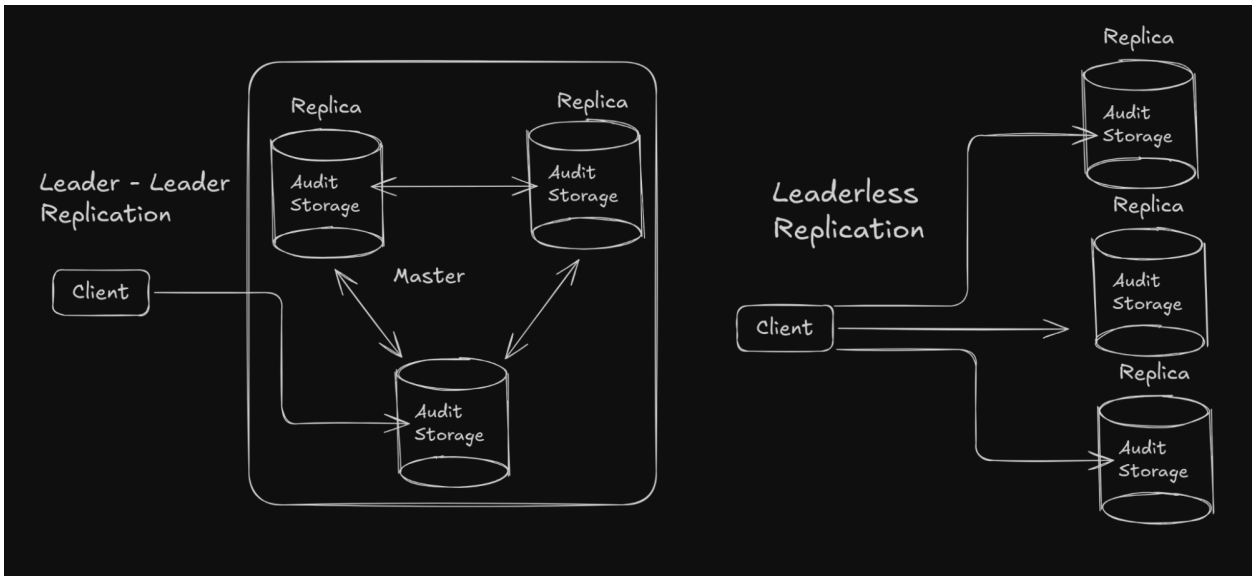
Using consensus here would:

- increase latency,
- increase operational complexity,
- reduce write throughput unnecessarily.

# 8.22.5 Suitable Replication Strategies

Instead, Audit Storage can use:

- Leader-Leader replication
  or
- Leaderless replication




These architectures prioritize:

- durability,
- availability,
- write scalability.

# 8.22.6 Leader-Leader Replication

In:

```
None
Leader-Leader replication
```

multiple nodes can accept writes simultaneously.

Benefits:

- high availability
- no single write bottleneck
- regional write support
- fast failover

Nodes asynchronously synchronize updates with one another.

# 8.22.7 Leaderless Replication

Another suitable option is:

```
None
Leaderless replication
```

similar to DynamoDB/Cassandra-style systems.

Clients write to multiple replicas directly.

Advantages:

- no leader bottleneck
- better fault tolerance
- improved availability
- simpler failover behavior

Durability is achieved through:

```
None
Quorum acknowledgments
```

Example:

```
None
Write to 3 replicas

Require 2 acknowledgments
```

This ensures audit entries survive failures.

# 8.22.8 Why Conflicts Are Rare

Normally multi-writer systems introduce conflict resolution challenges.

However Audit Storage has a very important property:

```
None
Audit records are append-only
```

Additionally:

- workers process ordered payment events,
- updates for the same payment are serialized through Kafka partitions.

Therefore concurrent conflicting writes are extremely uncommon.

This makes:

- leader-leader,
- and leaderless systems

much easier to operate safely.

# 8.22.9 No Business Decisions Depend on Audit Reads

Another important observation:

```
None
The system does not make live payment decisions from Audit Storage
```

Meaning:

- stale reads are acceptable,
- temporary inconsistency is acceptable,
- replica lag is acceptable.

Audit Storage exists primarily for:

- durability,
- compliance,
- historical reconstruction.

# 8.22.10 Replay and Recovery

Because audit logs are immutable:

```
None
They become a source for system recovery
```

The system can:

- replay audit entries,
- reconstruct merchant views,
- rebuild analytics systems,
- debug operational incidents.

This makes Audit Storage a foundational reliability layer.

# 8.22.11 CAP Tradeoff

Audit Storage typically behaves closer to:

```
AP-oriented durable storage
```

under CAP theorem.

It prioritizes:

- availability,
- durability,
- partition tolerance

while relaxing:

- strict consistency,
- recency guarantees.

This is acceptable because:

- correctness decisions are handled elsewhere,
- payment execution relies on Payment Storage,
- not on audit replicas.

# 8.22.12 Architectural Insight

Audit Storage demonstrates an important distributed systems principle:

```
Different workloads require different consistency guarantees
```

| Component | Consistency Requirement |
| --- | --- |

| | |
|---|---|
| Payment Storage | Strong consistency |
| Merchant Storage | Eventual consistency |
| Audit Storage | Durable eventual consistency |

By tailoring replication strategy to workload characteristics, the system achieves:

- scalability,
- reliability,
- compliance,
- and operational efficiency

without overengineering every storage layer with the same consistency model.

# 8.23 Payment Events Queue

The system uses:

```
None
Kafka
```

as the central:

```
None
PaymentEvents Queue
```

for asynchronous event processing.

# 8.23.1 Why Introduce a Queue

Payment processing generates many independent side effects:

- webhook delivery,
- audit logging,
- merchant storage synchronization,
- notifications,
- reconciliation jobs,
- analytics pipelines.

These operations are:

```
None
Independent and failure-prone
```

Therefore:

```
None
Asynchronous event-driven processing
```

becomes essential for scalability and reliability.

# 8.23.2 Kafka Partitioning Strategy

Kafka topics are:

```
None
Partitioned by payment_id
```

This guarantees:

```
None
Ordering within the same payment
```

For example:

```
None
created → authorized → captured
```

will always be processed sequentially for a specific payment.

# 8.23.3 Why Ordering Matters

Payment state transitions are highly order-sensitive.

Incorrect ordering may create invalid states such as:

```
None
captured → authorized
```

which violates payment correctness.

Partitioning by:

```
None
payment_id
```

ensures all events for the same payment land on:

```
None
The same Kafka partition
```

therefore preserving event order.

# 8.23.4 Kafka Scalability

Kafka naturally scales horizontally by:

```
None
Adding more partitions
```

This allows:

- parallel processing,
- higher throughput,
- distributed consumers,
- and independent scaling of workloads.

# 8.23.5 Replication and Availability

To guarantee durability:

```
None
Kafka partitions are replicated
```

across brokers.

Using:

- replication factor,
- ISR (in-sync replicas),
- and quorum acknowledgements,

Kafka can provide:

```
None
High availability without losing events
```

# 8.23.6 Quorum-Based Durability

Writes are acknowledged only after:

```
None
A quorum of replicas confirms persistence
```

This prevents data loss during:

- broker crashes,
- network failures,
- or leader failover.

# 8.23.7 At-Least-Once Delivery

The queue operates with:

```
None
At-least-once delivery semantics
```

Meaning:

- events are never silently lost,
- but duplicate processing is possible.

This is acceptable because downstream systems are designed to be:

```
None
Idempotent
```

# 8.23.8 Backpressure Handling

Kafka also acts as:

```
None
A buffering layer
```

during traffic spikes.

If downstream workers slow down:

- events accumulate in partitions,
- producers continue operating,
- consumers catch up later.

This decouples:

```
None
Traffic spikes from processing speed
```

# 8.23.9 Fault Isolation

Failures in one side effect pipeline:

- webhook delivery,
- analytics,
- merchant synchronization,

do not block:

```
None
Core payment processing
```

This significantly improves:

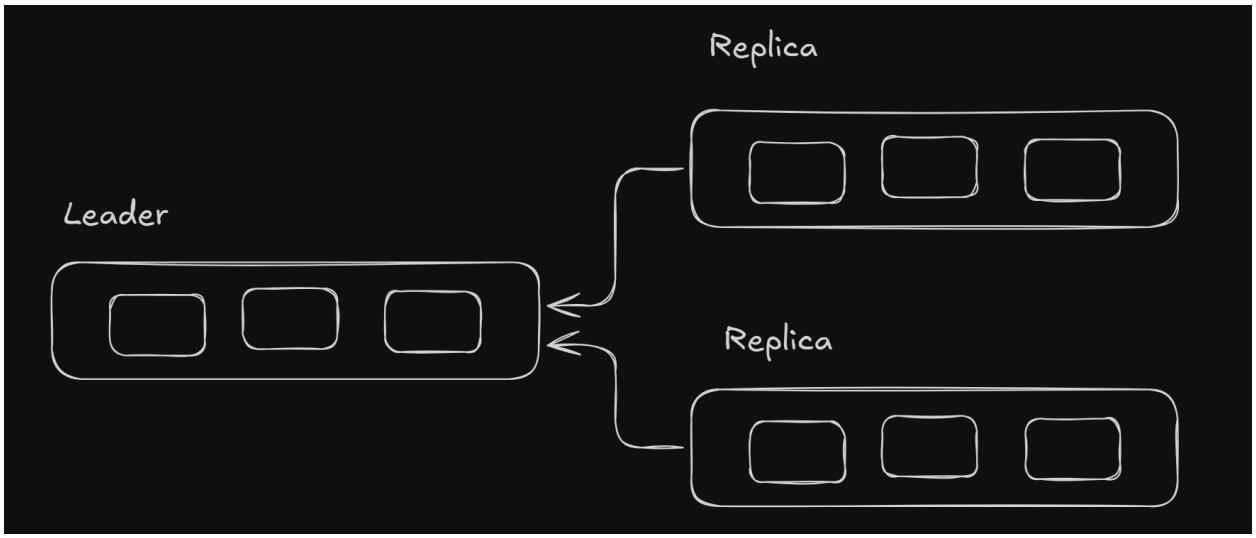
- resiliency,
- isolation,
- and overall availability.

# 8.24 Workers and Background Jobs

Workers consume events from:

```
None
PaymentEvents Queue
```

and execute asynchronous side effects.




# 8.24.1 Responsibilities of Workers

Typical worker responsibilities include:

- sending webhooks,
- updating Merchant Storage,
- writing Audit logs,
- triggering notifications,
- reconciliation processing,
- retry execution.

# 8.24.2 Stateless Worker Design

Workers are intentionally designed as:

```
None
Stateless compute nodes
```

They do not store local session state.

All durable state lives in:

- Kafka,
- Retry Storage,
- or databases.

# 8.24.3 Horizontal Scalability

Because workers are stateless:

```
None
Scaling is trivial
```

We simply:

- add more worker instances,
- assign more Kafka partitions,
- distribute consumption load.

# 8.24.4 Worker Parallelism

Kafka partitions naturally distribute work across consumers.

This enables:

```
None
Massive parallel event processing
```

while still preserving:

```
None
Per-payment ordering guarantees
```

# 8.24.5 Failure Recovery

If a worker crashes:

- Kafka retains the event,
- offsets are not committed,
- another worker reprocesses it.

This creates:

```
None
Reliable failure recovery
```

without manual intervention.

# 8.24.6 Duplicate Processing

Because of:

```
None
At-least-once delivery
```

workers may occasionally process the same event multiple times.

Therefore worker operations must be:

```
None
Idempotent
```

Example:

- webhook deduplication,
- UPSERT operations,
- unique event identifiers,
- conditional writes.

# 8.24.7 Independent Scaling

Different worker groups may scale independently:

| Worker Type | Scaling Need |
|---|---|
| Webhook workers | High network |
| Analytics workers | CPU-heavy |
| Audit workers | Write-heavy |
| Notification jobs | Burst-heavy |

This provides:

```
None
Operational flexibility
```

# 8.24.8 Regional Deployment

Workers may also be deployed across:

- multiple regions,
- multiple availability zones,
- or multiple clusters.

This improves:

- fault tolerance,
- disaster recovery,
- and latency.

# 8.25 Retry / Job Storage

Retry Storage maintains:

```
None
Durable background job state
```

for operations requiring retries.

# 8.25.1 Why Retry Storage Exists

Certain operations may fail temporarily:

- merchant webhooks,
- external APIs,
- payout processing,
- notification systems.

These failures require:

```
None
Persistent retry scheduling
```

rather than in-memory retries.

# 8.25.2 Stored Job Information

Typical retry job fields include:

- job_id,
- job_type,
- retry_count,
- execution_time,
- payload,
- status.

Example:

```JSON
{

  "job_id": "j123",

  "type": "webhook_retry",

  "status": "pending"

}
```

# 8.25.3 Sharding Strategy

Retry Storage can be:

```None
Sharded by job_id
```

because job identifiers distribute evenly.

This enables:

- scalable writes,
- balanced storage utilization,
- and parallel retry execution.

# 8.25.4 Query Patterns

Background systems may additionally query jobs by:

```
None
type
```

for example:

- all pending webhook retries,
- all failed payouts,
- all reconciliation jobs.

To support this efficiently:

```
None
Secondary indexes
```

can be added.

# 8.25.5 Availability Requirements

Retry Storage operates with:

```
None
At-least-once guarantees
```

Therefore:

```
None
Data loss is unacceptable
```

because lost jobs may permanently skip critical retries.

# 8.25.6 Consistency Requirements

Unlike Payment Storage:

```
None
Retry Storage does not require linearizability
```

because retry execution is asynchronous.

Temporary inconsistency is acceptable as long as:

```
None
Jobs are eventually executed
```

# 8.25.7 Replication Strategy

Suitable replication models include:

- Leader-Leader replication
- Leaderless replication
- Quorum-based replication

These provide:

- high availability,
- durability,
- and fault tolerance.

# 8.25.8 Why Leaderless Works Well

Retry systems are naturally tolerant of:

- delayed visibility,
- duplicate execution,

- and eventual synchronization.

Therefore:

```
None
Strict consensus is unnecessary
```

making:

```
None
Leaderless architectures practical
```

for background job systems.

# 8.26 Conclusion

We started from a very small:

```
None
Single-node payment system
```

and gradually evolved it into a:

```
None
Stripe-scale distributed architecture
```

capable of handling:

- massive traffic,
- high availability,
- strong durability guarantees,
- and operational scalability.

# 8.26.1 Core Architectural Evolution

The system evolved through multiple stages:

| Stage | Improvement |
| --- | --- |
| Basic API | Simple payment processing |
| Kafka introduction | Asynchronous side effects |
| Dedicated storages | Workload isolation |
| Sharding | Horizontal scalability |
| Replication | High availability |
| Consensus systems | Strong correctness guarantees |

Each step solved a specific bottleneck in:

```
None
Correctness, scalability, or reliability
```

# 8.26.2 Separation of Responsibilities

The final architecture separates concerns cleanly:

| Component | Responsibility |
| --- | --- |
| Payment API | Core payment orchestration |
| Payment Storage | Transactional payment state |
| Merchant Storage | Merchant-facing queries |
| Audit Storage | Immutable financial ledger |
| Kafka Queue | Event distribution |
| Workers | Async side effects |
| Retry Storage | Durable retries/background jobs |

This separation prevents:

```
None
Mixed workload interference
```

and allows every subsystem to optimize for:

```
None
Its own access patterns
```

# 8.26.3 Scalability Principles

The system achieves scalability through:

- stateless services,
- horizontal scaling,
- sharding,
- partitioned queues,
- asynchronous processing,
- and workload isolation.

Critical scaling strategies include:

| Component | Scaling Strategy |
|---|---|
| APIs | Stateless replication |
| Kafka | Partition scaling |
| Payment Storage | Shard by payment_id |
| Merchant Storage | Shard by merchant_id |
| Audit Storage | Append-heavy partitioning |
| Workers | Consumer parallelism |

# 8.26.4 Consistency Decisions

Different components require different consistency models.

| Component | Consistency Requirement |
| --- | --- |
| Payment Storage | Strong consistency / linearizability |
| Merchant Storage | Eventual consistency |
| Audit Storage | Durable eventual consistency |
| Retry Storage | At-least-once durability |

This demonstrates an important distributed systems principle:

```
None
Not every subsystem needs the same consistency guarantees
```

Choosing the proper tradeoff improves:

- performance,
- availability,
- and operational simplicity.

# 8.26.5 Availability and Fault Tolerance

To remain highly available:

- APIs are replicated across regions,
- Kafka uses replicated partitions,
- workers are stateless,
- databases use replication strategies appropriate to their workloads.

The architecture tolerates:

- machine failures,
- broker failures,
- worker crashes,
- network partitions,
- and temporary downstream outages.

# 8.26.6 Event-Driven Architecture

One of the most important improvements was introducing:

```
None
Kafka-based asynchronous event processing
```

This decouples:

- payment execution,
- webhook delivery,
- audit logging,
- analytics,
- and merchant synchronization.

Benefits include:

- retryability,
- independent scaling,
- fault isolation,
- and resilience under spikes.

# 8.26.7 Data Correctness

Financial systems require extremely strong guarantees.

Therefore the architecture prioritizes:

```
None
Never losing payment or audit data
```

using:

- consensus replication,
- quorum writes,
- append-only ledgers,
- and durable retry systems.

At the same time:

```
None
Eventually consistent systems
```

are used where business requirements allow them.

# 8.26.8 Real-World Engineering Tradeoffs

A key insight from this design is:

```
None
Distributed systems are about tradeoffs
```

There is no universally perfect solution.

Every design decision balances:

- consistency,
- availability,
- latency,
- complexity,
- scalability,
- and cost.

For example:

| Decision | Tradeoff |
| --- | --- |
| Consensus replication | Strong correctness but higher latency |
| Async processing | Better scalability but eventual consistency |
| Merchant Storage | Faster reads but duplicated data |
| At-least-once delivery | Reliability but duplicate handling |

# 8.26.9 Final Architectural Insight

At large scale, successful payment systems rely heavily on:

```
None
Specialized distributed subsystems
```

rather than one universal database or service.

The architecture becomes:

- highly decoupled,
- event-driven,
- horizontally scalable,
- and failure-tolerant.

This is what enables systems like:

- Stripe,
- PayPal,
- Adyen,
- or Square

to process:

```
Millions of financial transactions reliably every day
```

while maintaining:

- correctness,
- durability,
- and high availability.